

University of Engineering & Technology

Peshawar

Computer Programming (C++) Lab

Lab 12

STUDENT NAME	Muhammad Abdal Ahmad
REGISTRATION NO:	BF25NWELE0701
COURSE:	EE-170L COMPUTER PROGRAMMING LAB
DATE:	14/05/26
Lab Title:	POINTERS IN C++

SUBMITTED TO ENGR MUHAMMAD RIZWAN

1 Objective:

The objective of this lab session is to introduce and apply the following concepts in C++:

- Computer Memory and Variables
- Pointers
- Pointer Declaration
- Referencing Operator (&)
- Dereferencing Operator (*)

2 Computer Memory:

The computer's memory is made up of bytes. Each byte has a unique address associated with it. When a program runs, variables are stored in memory at specific addresses determined by the compiler. The size of the memory block reserved depends on the data type: for example, on a 32-bit PC an integer occupies 4 bytes, while on older 16-bit PCs integers occupied 2 bytes.

2.1 Variables and Memory:

When a variable is declared, the compiler reserves a specific block of memory to hold its value. Two distinct steps occur when a variable is accessed:

- The program locates the address reserved for the variable.
- The contents stored at that address are retrieved.

```
#include <iostream>
using namespace std;
int main() {
    float f1 = 3.14;
    cout << f1;
    return 0;
}
```

Output

3.14

Note: The float value 3.14 is actually stored as 32 ones and zeros (4 bytes × 8 bits) in memory, not as individual characters.

3 Pointers in C++:

A pointer is a variable that holds the memory address of another variable. Unlike regular variables that store values directly, a pointer stores the address where a value is kept. Because a pointer holds an address rather than a value, it has two components:

- The pointer variable itself (holds the address).
- The value stored at that address.

3.1 Pointer Declaration:

The syntax for declaring a pointer variable is:

```
// Syntax: dataType *variableName;
int *ptr;           // Declare a pointer to an integer
float *fptr;        // Declare a pointer to a float
char *cptr;         // Declare a pointer to a character
```

The asterisk (*) tells the compiler that a pointer variable is being created. The data type specifies the type of data at the address the pointer will hold.

3.2 Referencing Operator (&):

The address-of operator (&) retrieves the memory address of a variable and assigns it to a pointer:

```
int *ptr;
int k;
ptr = &k;    // ptr now holds the address of k
```

After this statement, ptr is said to "point to" k.

3.3 Dereferencing Operator (*):

The dereference operator (*) accesses the value at the address stored in a pointer:

```
*ptr = 7;        // Set the value the pointer points to, to 7
cout << *ptr;    // Print the value stored at the address ptr points to
```

When * is used in this context, we are referring to the value that ptr points to, not the pointer's own address value.

3.4 Example: Pointer Operations:

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int b = 15;
    int *p1, *p2;
    p1 = &a;
    p2 = &b;
    *p1 = 10;
    *p2 = *p1;
    *p1 = 20;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    return 0;
}
```

Output

a = 20

b = 10

4 Lab Tasks — Lab 12:

Task 1: Pointer Declaration and Initialization:

Declare an integer variable num, a pointer ptr, initialize the pointer to point to num, and display the values using the dereference operator.

```
#include <iostream>
using namespace std;
int main() {
    int num = 42;
    int *ptr;
    ptr = &num;
    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Value of ptr (address): " << ptr << endl;
    cout << "Value pointed by ptr: " << *ptr << endl;
    return 0;
}
```

Output

Value of num: 42

Address of num: 0x61ff08

Value of ptr (address): 0x61ff08
Value pointed by ptr: 42

Task 2: Pointer Arithmetic:

Declare an integer array, use a pointer to traverse and display its elements using pointer arithmetic.

```
#include <iostream>
using namespace std;
int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int *ptr = numbers;
    int size = 5;
    cout << "Array elements using pointer arithmetic:" << endl;
    for (int i = 0; i < size; i++) {
        cout << "numbers[" << i << "] = " << *(ptr + i) << endl;
    }
    return 0;
}
```

Output

Array elements using pointer arithmetic:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50